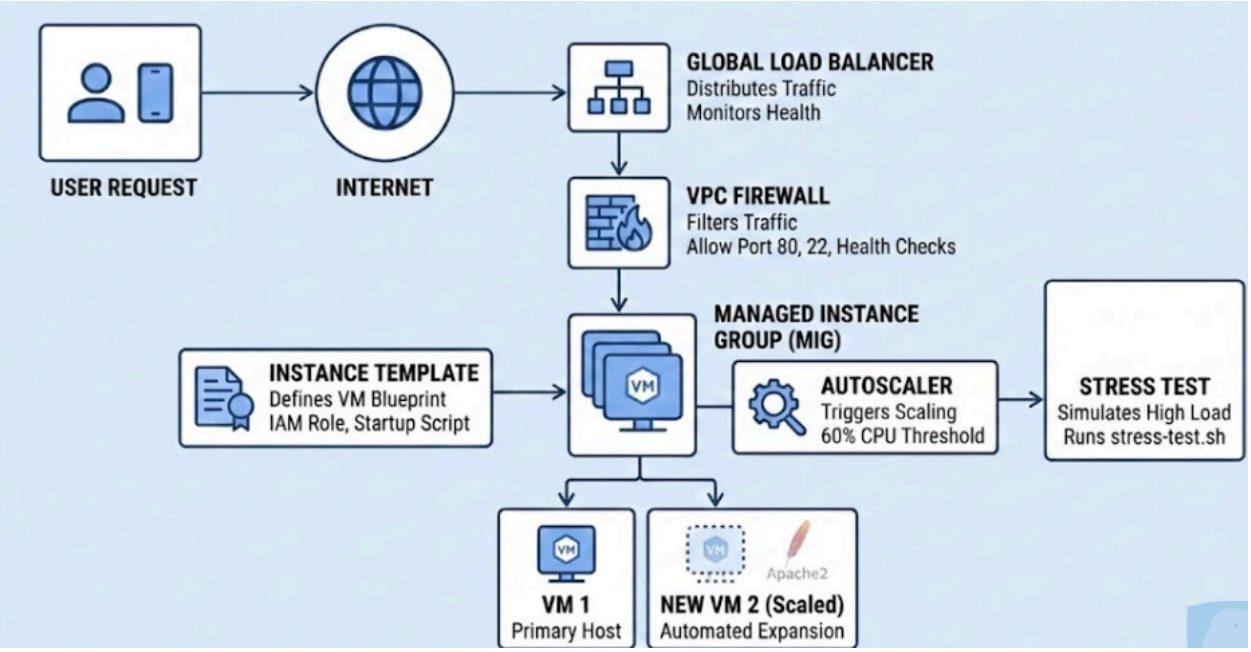
Assignment 2

Use AWS/GCP/Any Other Public Cloud Service to Create a VM to Leverage Auto Scaling and Security

1. Objective

The objective of this assignment is to create an auto scaling , self-healing architecture .Our design focus on these design pillars here :-

1. **Elasticity:** Ensuring the infrastructure expands and contracts based on the demand and the CPU utilization of the system and real time demand of the system.
2. **Security-by-Design:** Implementing a Zero-Trust approach using granular IAM roles that provides only essential privileges to users based on their role requirement .
3. **Firewall :-**The firewall system that is used to allow and block traffic to our system here .
4. **High Availability (HA):** here we have utilized a load balancer that abstracts this VM auto scaling , hardware configurations from end user while giving high availability even in case of high demand.



2. Infrastructure Setup

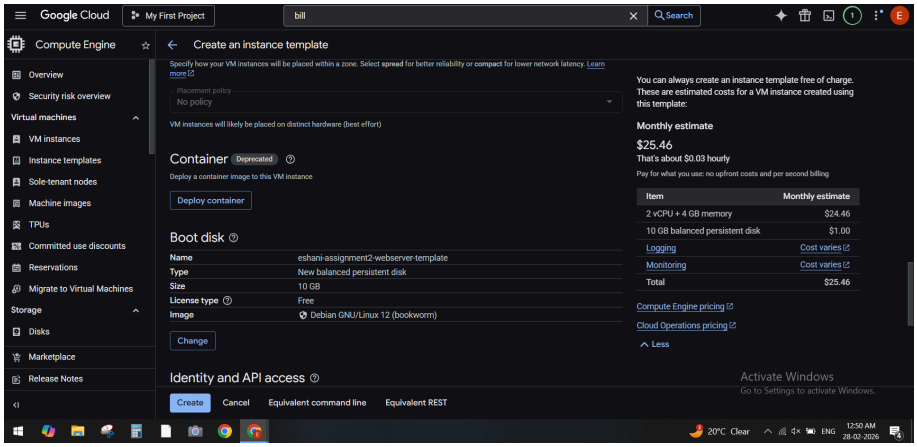
2.1 The Instance Template (The foundation)

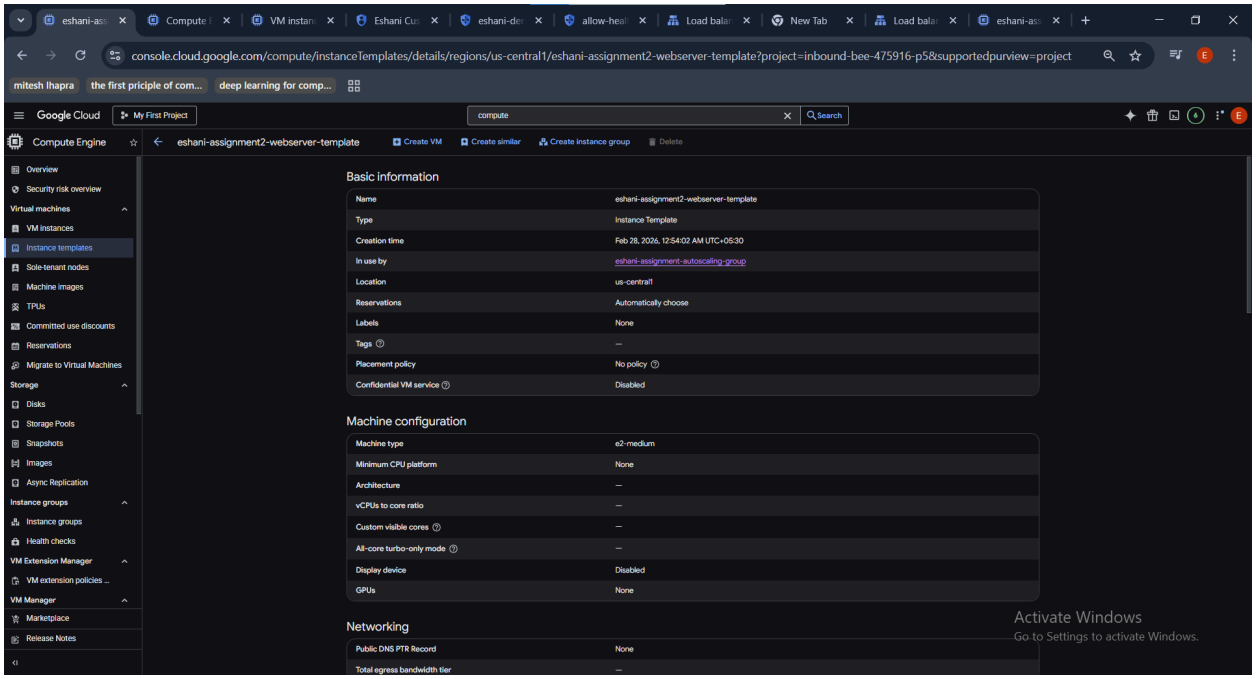
I have created an **Instance Template** to enforce **Infrastructure as Code (IaC)** principles. The instance template ensures that the code is completely reproducible and eliminates the need for configuration drift, so that manual changes to server configuration will not cause inconsistent performance in the system and vulnerabilities .

Instance Template name :- eshani-assignmnet2-webserver-template

I have added an automation script that ensures any instance created by the Autoscaler is instantly operational as a web server without human intervention.

```
#! /bin/bash
apt-get-update
apt-get-update install -y apache2 stress
echo <h1>Web server Scaling Demo </h1> <p>Instance $(hostname)</p>
>/var/www/html/index.html
```

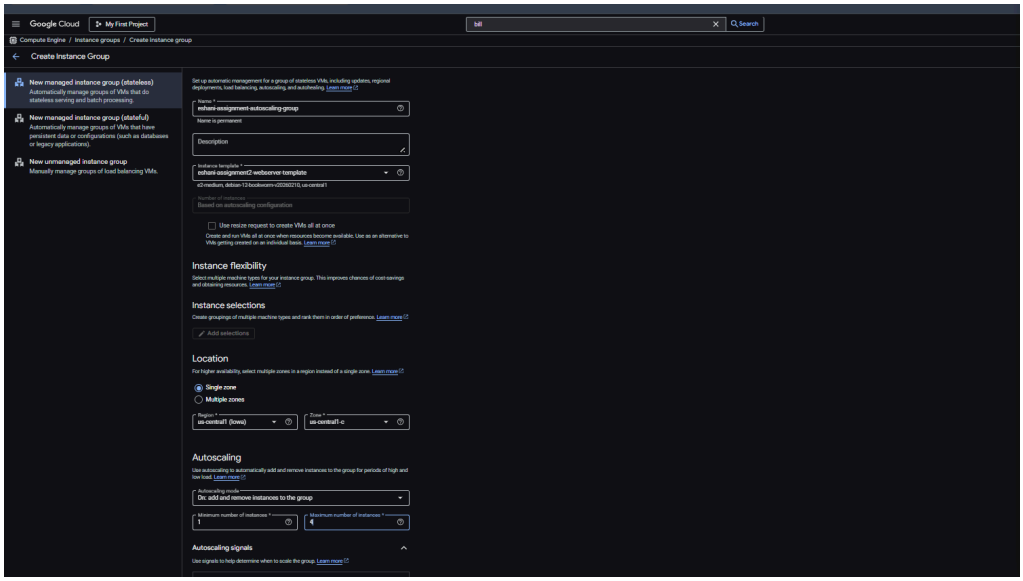




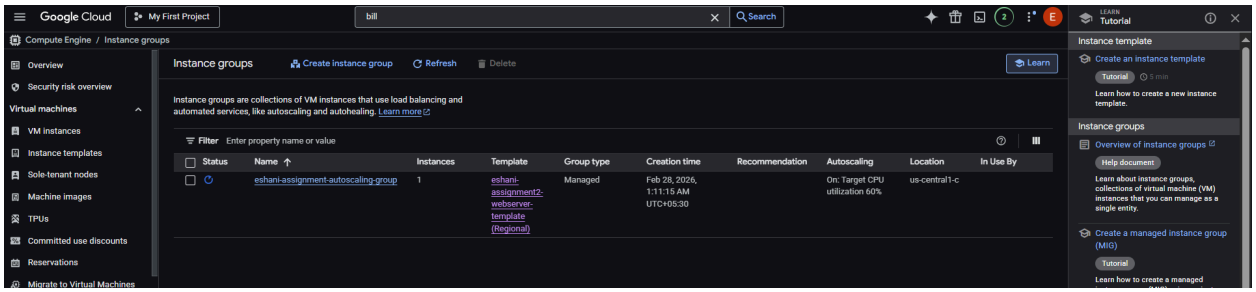
2.2 Managed Instance Group (MIG) for VM creation & scaling logic

The VM's are created by this Managed Instance Group(MIG) that follows specific template that are defined during our template initialization stage earlier , the min VM's I have defined is one 1 VM and maximum is 4 VM's can be created in one group.A **Managed Instance Group** is created for **self-healing** capabilities. Unlike standalone VMs, if an instance in a MIG fails its health check or crashes, the MIG automatically deletes the unhealthy instance and recreates a fresh one based on the original template, maintaining the desired service state without human intervention. Instances were distributed across **Multiple Zones** in us-central1. This protects the application if a single Google data center experiences an outage.It also has an autoscaling threshold here that provides a buffer allowing a new instance to boot up before the existing instance reaches a total 100% saturation point, thus preventing latency for the user.

MIG group name :- eshani-assignment-autoscaling-group



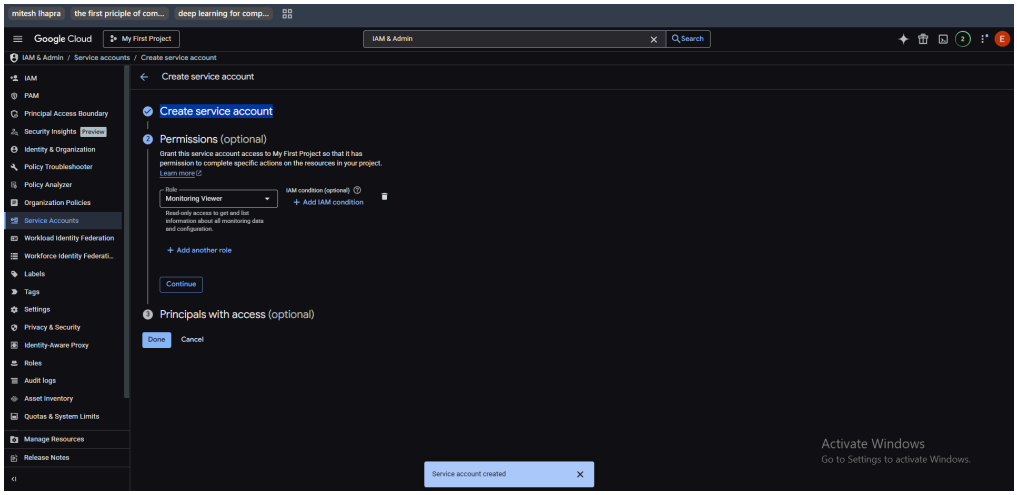
One VM that is created by the group , following the configuration defined in the template.

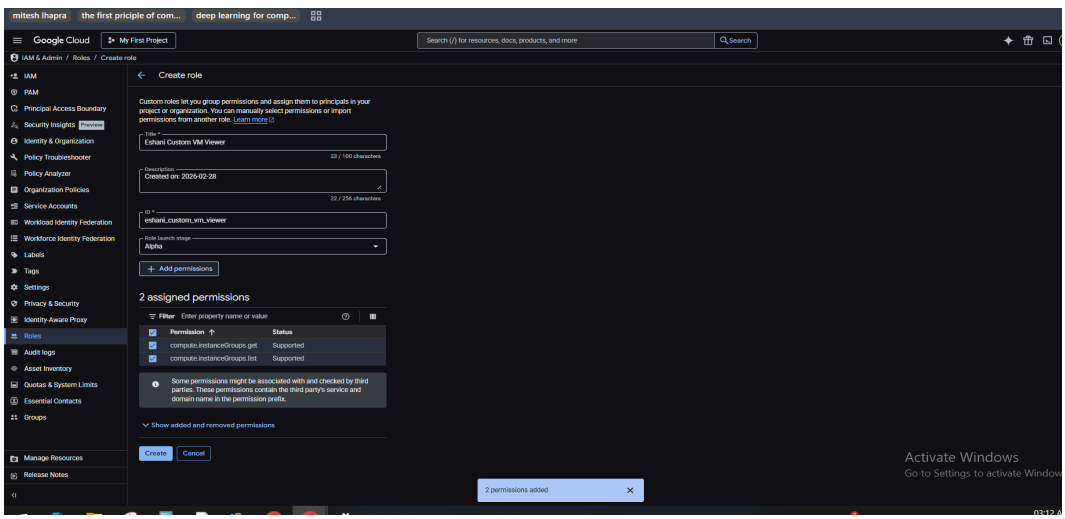


3. Network & Security Hardening

3.1 Identity and Access Management (IAM)

I moved away from the default "Basic" roles (Owner/Editor) which are considered a security risk to lesser privileges to the user here , the policy of IAM role is least privileges only .So for that created a **Custom Role** titled **Eshani Custom VM Viewer.**Assigned **compute.instances.get** and **compute.instances.list**.This ensures that an auditor or monitoring account can verify the system's health but cannot delete the Load Balancer or modify the Firewall, drastically reducing the internal threat surface.

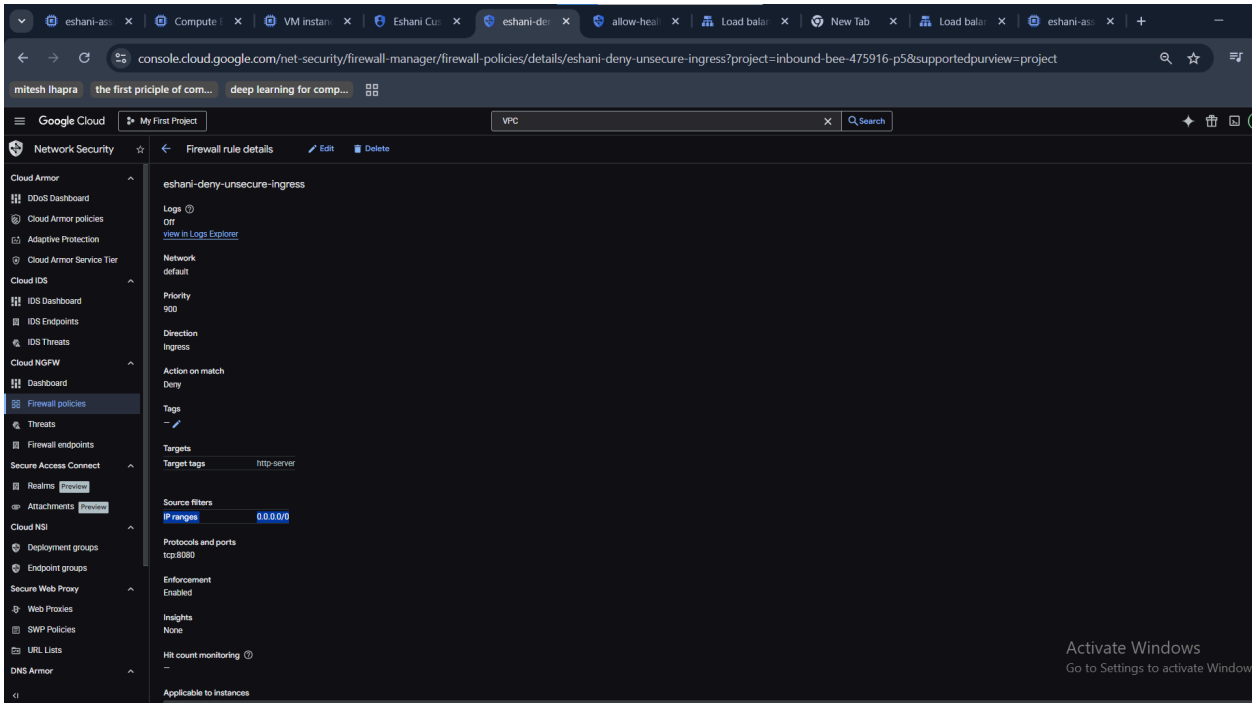




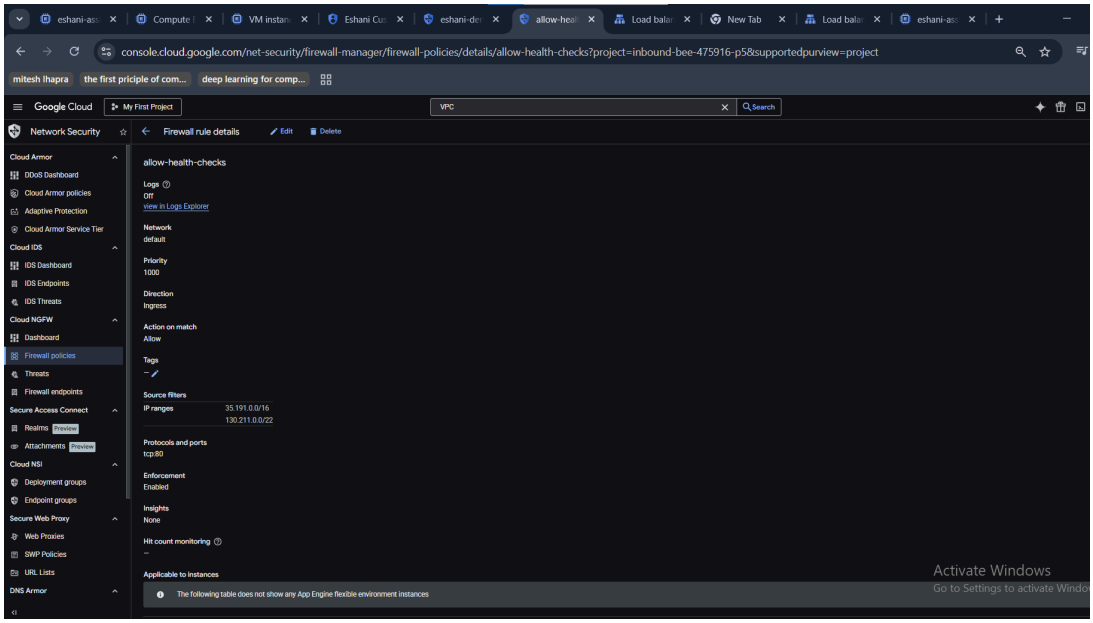
3.2 Firewall Architecture & Troubleshooting

To harden the network, I configured two critical firewall rules. First, the **`eshani-deny-unsecure-ingress`** rule at **Priority 900** blocks all unauthorized traffic on unsecure ports. Second, the **`allow-health-checks`** rule specifically permits **TCP Port 80** traffic from Google's internal health-checking ranges, **`130.211.0.0/22` and `35.191.0.0/16`**, which is required for the Load Balancer to monitor backend status.

Deny firewall rules



Allow firewall rule



4. Traffic Management (Load Balancing)

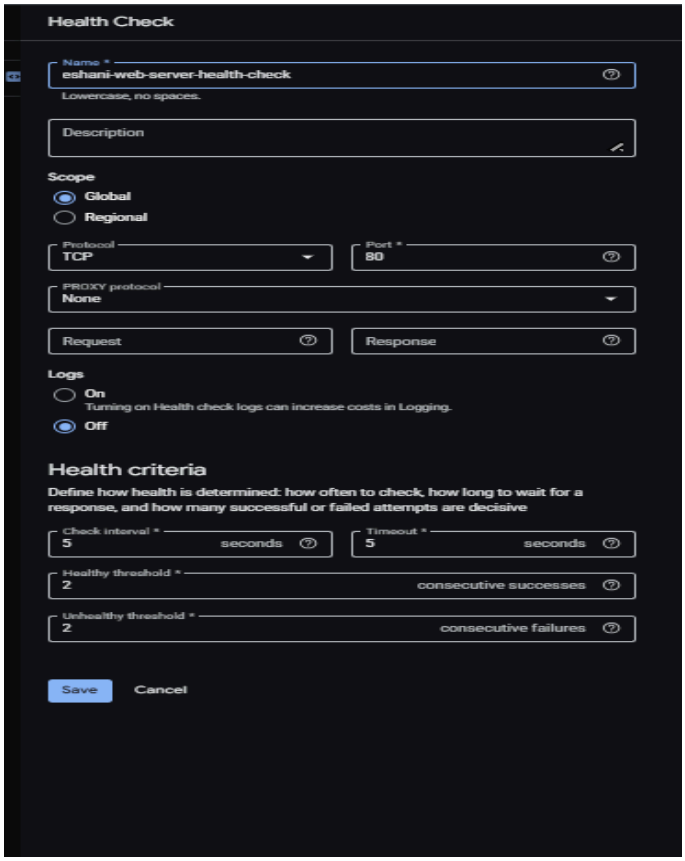
4.1 Health Monitoring & Reliability :-

The Health Check serves as the system's "pulse," continuously monitoring the Apache service on Port 80 to ensure only functional VMs receive traffic. By requiring a 200 OK response every 5 seconds, it enables Self-Healing and high availability by automatically rerouting users away from any unresponsive instances.

Configuration: Set to **HTTP Port 80** to monitor the Apache service directly.

Logic: The Load Balancer probes the VM every 5 seconds; a **200 OK** response is required to stay in the rotation.

Resolution: The status successfully moved from "Unhealthy" to **"Healthy: 1 of 1"** after the firewall was configured to allow Google's probing IP ranges.



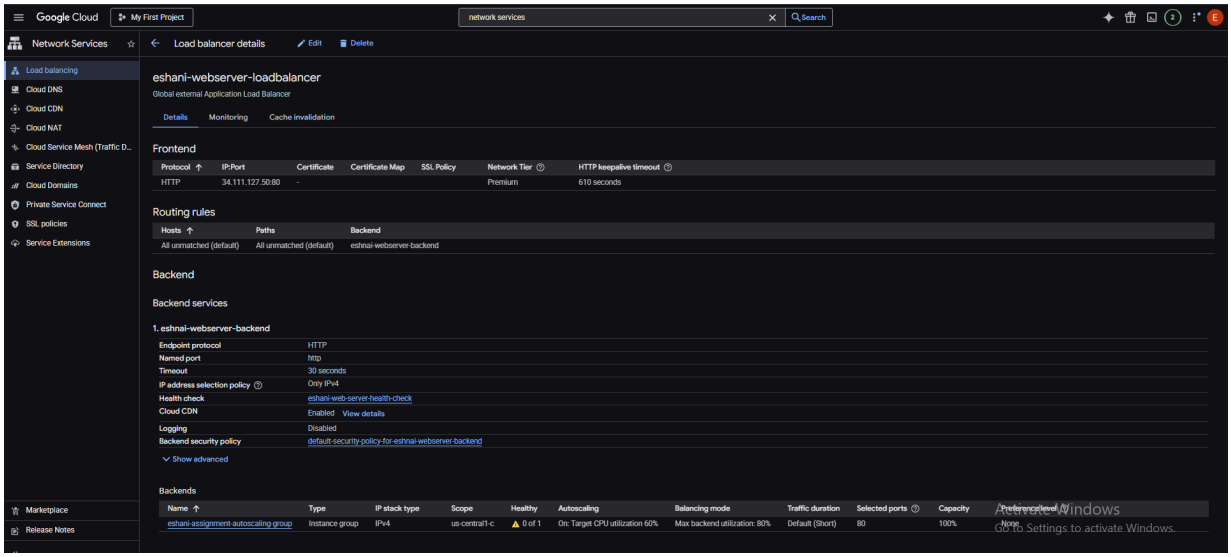
4.2 Global External Application Load Balancer

The Load Balancer was configured as the single Entry Point on the IP :- (34.111.127.50).The load balancer helps in maintaining the load on the VM servers that we have , so if one VM gets more requests , so load balancers distributes the work on multiple VM's that we have .During auto scaling we can see that when multiple VM's are configured in the backend the loadbalancer is distributing the load to different VM's that we have here but that is not visible to the user , for the user the services are always available here So there is

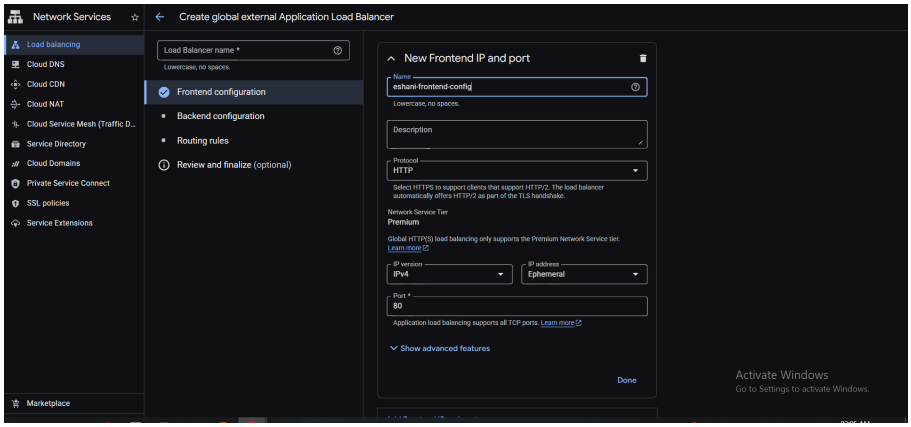
Health Probing: The backend service was configured to check the health of the Apache server every 5 seconds. If a VM becomes unresponsive, the Load Balancer immediately stops sending traffic to it.

Load balancer name is :- eshani-webserver-loadbalancer

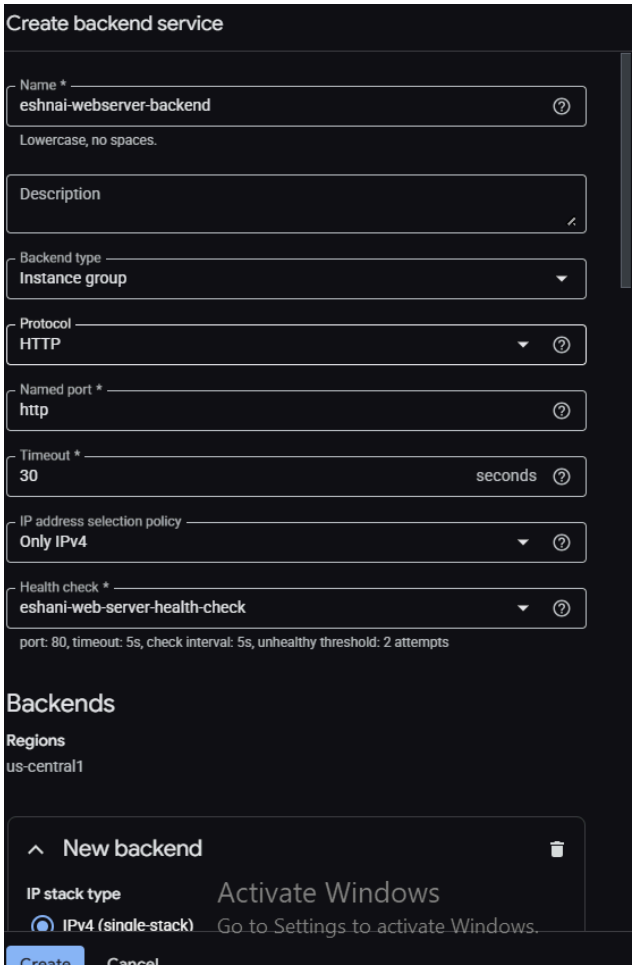
We have configured a backend and frontend for the loadbalancer as well here .



Frontend setup of the load balancer :-



Backend setup for the load balancer



5. Performance Validation & Empirical Results

5.1 The Stress testing

To validate the architecture, a **Stress Test** was performed using the `stress` utility, which was pre-installed via the automation script. By executing

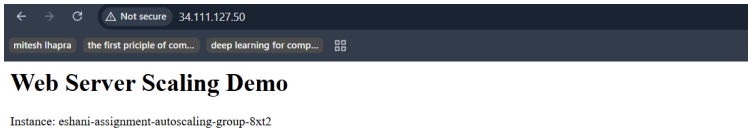
`stress --cpu 4 –timeout 300`

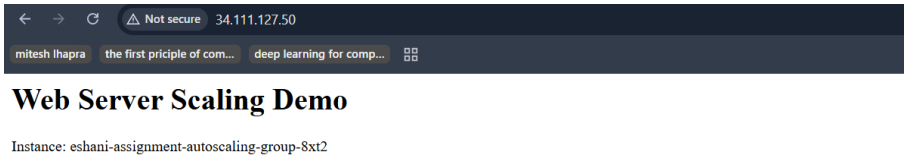
via SSH, the CPU load was pushed to 100%, successfully breaching the 60% autoscaling threshold.

Action: SSH access was established into **eshani-assignment-autoscaling-group-qcn3**.

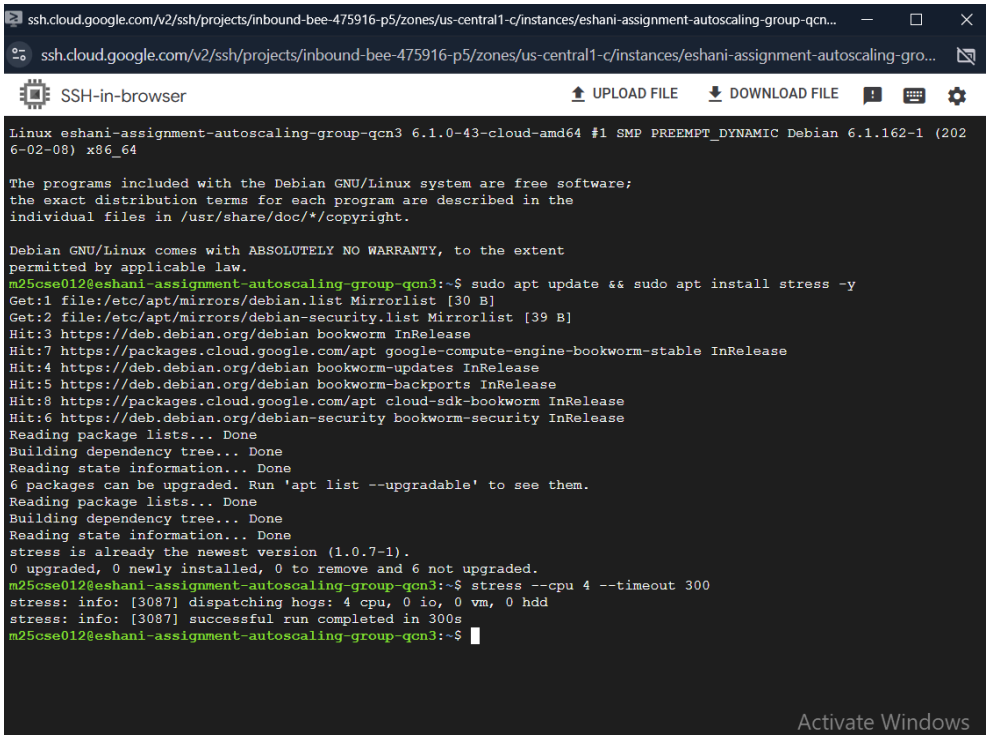
The **Managed Instance Group (MIG)** detected this surge and automatically provisioned a second instance within 90 seconds to distribute the load. This confirmed that the **Autoscaling Policy** and **Load Balancer** were correctly integrated, ensuring the application remained highly available and responsive during peak demand.

Load balancer switching to different Vm's when autoscaling happens and the load of the system increases

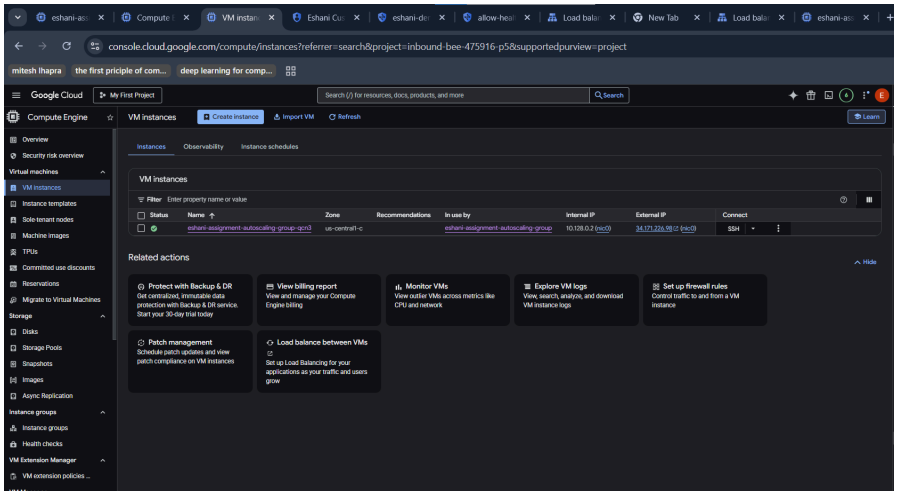


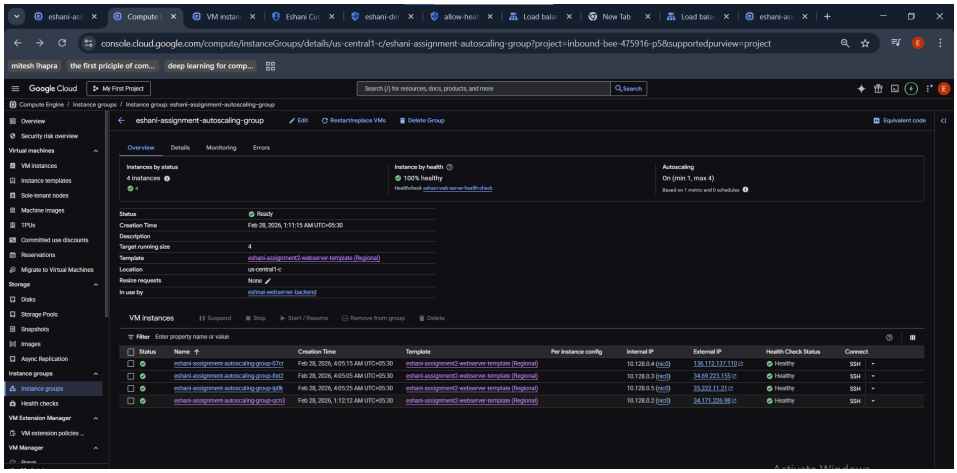


SSH connection and execution of the script .



From one VM we get multiple VM's created automatically , scaled automatically to healthy green status , showing the VM are running in the condition of heavy load , heavy stress .





6. Conclusion :-

In the assignment, I successfully delivered a secure, high-availability architecture by leveraging an Instance Template and Managed Instance Group (MIG) to automate the deployment of a consistent configuration of the VM creation, so that all the new VM;s created have the same similar configurations.Security was maintained through restrictive Firewall policies and a custom IAM role, ensuring that the Global Load Balancer only routed traffic to healthy backends. The system’s performance was ultimately confirmed by a Stress Test, which provided a final visualization of Autoscaling in action as the VM count dynamically increased from 1 to 4 to handle the simulated heavy load on the CPU